

Tài liệu học tập ANN3

Huấn luyện mạng nơ-ron, lan truyền ngược và hàm mất mát

Biên soạn theo vai trò trợ giảng chuyên môn

Ngày 9 tháng 7 năm 2026

Mục lục

Mục tiêu bài học

Bài ANN3 tập trung vào quá trình huấn luyện mạng nơ-ron nhân tạo. Sau khi học xong, người học cần nắm được:

1. Vì sao mạng nơ-ron ban đầu chưa có khả năng suy luận.
2. Ý nghĩa của việc khởi tạo ngẫu nhiên trọng số và bias.
3. Quá trình lan truyền tiến từ input đến output.
4. Ý tưởng tổng quát của lan truyền ngược sai số, tiếng Anh là *backpropagation*.
5. Vai trò của dữ liệu, nhãn, output dự đoán và sai số.
6. Cách biểu diễn nhãn phân loại bằng one-hot vector.
7. Một số cách đo sai số giữa hai vector: chuẩn L_2 , góc giữa hai vector và cross entropy.
8. Vì sao huấn luyện mạng nơ-ron là quá trình lặp nhiều lần và có thể tốn nhiều thời gian.

Chương 1

Nhắc lại bài toán học có giám sát

1.1 Mạng nơ-ron cần học ánh xạ

Trong các bài trước, ta đã biết mạng nơ-ron nhân tạo được dùng để xấp xỉ một hàm ánh xạ:

$$f : X \rightarrow Y$$

Trong đó:

- X là không gian dữ liệu đầu vào;
- Y là không gian kết quả đầu ra;
- f là quan hệ mà mô hình cần học.

Ví dụ:

Ảnh con chó $\rightarrow$ Nhãn “chó”

hoặc:

Giá cổ phiếu 5 ngày gần nhất $\rightarrow$ Giá cổ phiếu ngày tiếp theo

Ghi nhớ

Ban đầu, mạng nơ-ron chưa biết hàm f . Quá trình huấn luyện chính là quá trình điều chỉnh các tham số bên trong mạng để mạng học được ánh xạ từ input sang output.

1.2 Dữ liệu trong học có giám sát

Trong học có giám sát, tiếng Anh là *supervised learning*, ta cung cấp cho hệ thống nhiều cặp dữ liệu:

$$(x_i, y_i)$$

Trong đó:

- x_i là dữ liệu đầu vào, còn gọi là input hoặc data;
- y_i là nhãn đúng, còn gọi là label hoặc target.

Tập dữ liệu huấn luyện có dạng:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

Mục tiêu của mô hình là tạo ra dự đoán:

$$\hat{y}_i = f(x_i)$$

sao cho $\hat{y}_i$ càng gần y_i càng tốt.

Ví dụ minh họa

Bài toán phân loại ảnh gồm 4 lớp: chó, mèo, điều và xe hơi.

Mẫu dữ liệu	Input	Label đúng
1	Ảnh con chó	Chó
2	Ảnh con mèo	Mèo
3	Ảnh con điều	Điều
4	Ảnh xe hơi	Xe hơi

Mạng nơ-ron phải học từ nhiều ảnh đã được gán nhãn. Sau khi huấn luyện, khi gặp ảnh mới, mạng dự đoán ảnh thuộc lớp nào.

Chương 2

Thông số ban đầu của mạng nơ-ron

2.1 Tham số của từng nơ-ron

Một nơ-ron nhân tạo nhận nhiều đầu vào:

$$x_1, x_2, \dots, x_n$$

Mỗi đầu vào có một trọng số tương ứng:

$$w_1, w_2, \dots, w_n$$

Ngoài ra, nơ-ron còn có bias b . Giá trị trước hàm kích hoạt là:

$$z = \sum_{i=1}^n w_i x_i + b$$

Output của nơ-ron là:

$$a = \varphi(z)$$

Trong đó φ là hàm kích hoạt.

2.2 Mạng có rất nhiều tham số

Khi nhiều nơ-ron được kết nối thành một mạng, mỗi nơ-ron có bộ trọng số và bias riêng. Tổng số tham số trong mạng có thể rất lớn.

Ví dụ, nếu một nơ-ron ở lớp hiện tại nhận kết nối từ 5 nơ-ron ở lớp trước, thì nó có:

$$5 \text{ trọng số} + 1 \text{ bias} = 6 \text{ tham số}$$

Nếu mạng có hàng triệu kết nối, tổng số tham số có thể lên đến vài chục triệu hoặc vài trăm triệu.

Ghi nhớ

Huấn luyện mạng nơ-ron thực chất là tìm bộ giá trị phù hợp cho tất cả các trọng số và bias trong mạng.

2.3 Khởi tạo ngẫu nhiên

Khi bắt đầu huấn luyện, mạng chưa biết gì về bài toán. Vì vậy, các trọng số và bias thường được khởi tạo bằng các giá trị ngẫu nhiên.

Ký hiệu bộ tham số của mạng là:

$$\theta = \{W, b\}$$

Ban đầu:

$$\theta = \theta_0$$

Trong đó θ_0 là bộ tham số khởi tạo ngẫu nhiên.

Lưu ý

Trong thực tế, việc khởi tạo không phải là ngẫu nhiên hoàn toàn tùy tiện. Có nhiều kỹ thuật khởi tạo đặc biệt để giúp quá trình huấn luyện ổn định hơn. Tuy nhiên, ở mức nhập môn, ta có thể hiểu đơn giản rằng mạng bắt đầu từ một trạng thái ngẫu nhiên.

2.4 Vì sao output ban đầu thường sai?

Do các trọng số ban đầu là ngẫu nhiên, nên khi đưa input vào mạng, output cũng thường là một kết quả không có ý nghĩa.

Ví dụ minh họa

Giả sử đưa ảnh con chó vào mạng.

$$x = \text{ảnh con chó}$$

Label đúng là:

$$y = \text{chó}$$

Nhưng vì mạng mới khởi tạo ngẫu nhiên, nó có thể dự đoán:

$$\hat{y} = \text{máy bay}$$

Kết quả này sai vì mạng chưa được huấn luyện.

Chương 3

Lan truyền tiến

3.1 Ý tưởng của lan truyền tiến

Lan truyền tiến, tiếng Anh là *forward propagation*, là quá trình đưa dữ liệu từ lớp input đi qua các lớp ẩn và cuối cùng sinh ra output.

$$\text{Input} \rightarrow \text{Lớp ẩn 1} \rightarrow \text{Lớp ẩn 2} \rightarrow \dots \rightarrow \text{Output}$$

Ở mỗi nơ-ron, ta tính:

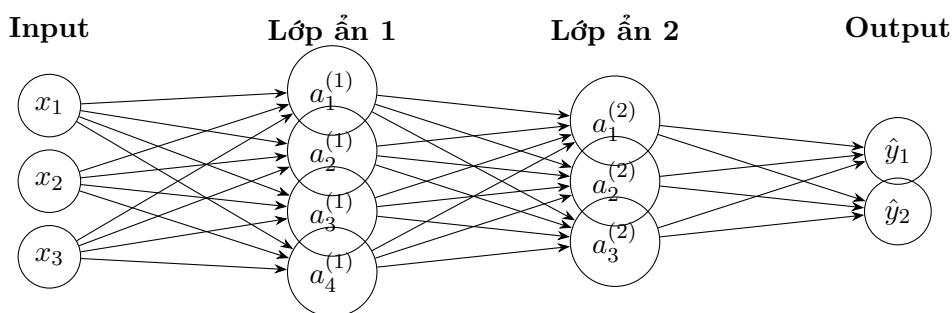
$$z = \sum_{i=1}^n w_i x_i + b$$

sau đó đưa qua hàm kích hoạt:

$$a = \varphi(z)$$

Output của lớp trước trở thành input của lớp sau.

3.2 Sơ đồ lan truyền tiến



3.3 Lan truyền tiến qua nhiều lớp

Giả sử mạng có L lớp. Với lớp thứ l , ta có:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \varphi(z^{(l)})$$

Trong đó:

- $a^{(0)} = x$ là input ban đầu;
- $W^{(l)}$ là ma trận trọng số của lớp l ;
- $b^{(l)}$ là vector bias của lớp l ;
- $a^{(l)}$ là output của lớp l ;
- output cuối cùng của mạng là $\hat{y} = a^{(L)}$.

Ví dụ minh họa

Xét một mạng đơn giản:

$$x \rightarrow \text{lớp ẩn} \rightarrow \hat{y}$$

Input:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Giả sử lớp ẩn có 2 nơ-ron, dùng hàm kích hoạt tuyến tính để tính đơn giản:

$$W^{(1)} = \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Ta tính:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

$$z^{(1)} = \begin{bmatrix} 1 & 0 \\ 0.5 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 3.5 \end{bmatrix}$$

Nếu hàm kích hoạt là tuyến tính, output lớp ẩn là:

$$a^{(1)} = \begin{bmatrix} 1 \\ 3.5 \end{bmatrix}$$

Chương 4

Sai số và ý tưởng huấn luyện

4.1 So sánh output dự đoán với output mong muốn

Sau khi lan truyền tiến, mạng tạo ra output dự đoán:

$$\hat{y}$$

Trong học có giám sát, ta biết output đúng:

$$y$$

Sai số được hiểu là sự khác biệt giữa $\hat{y}$ và y .

Sai số = khác biệt giữa output dự đoán và label đúng

Ví dụ minh họa

Nếu ảnh đầu vào là ảnh con chó, label đúng là:

$$y = \text{chó}$$

Nhưng mạng dự đoán:

$$\hat{y} = \text{máy bay}$$

Thì mạng đã sai. Ta cần đo mức độ sai đó bằng một hàm mất mát.

4.2 Hàm mất mát

Hàm mất mát, tiếng Anh là *loss function*, là hàm dùng để đo sai số giữa output dự đoán và output mong muốn.

$$\mathcal{L}(\hat{y}, y)$$

Hàm mất mát phải trả về một số vô hướng:

$$\mathcal{L} \in \mathbb{R}$$

Số này càng nhỏ thì dự đoán càng gần với nhãn đúng.

Ghi nhớ

Hàm mất mát biến khái niệm “mô hình dự đoán sai” thành một con số cụ thể. Nhờ đó, thuật toán huấn luyện có thể biết cần điều chỉnh tham số theo hướng nào.

4.3 Huấn luyện là giảm sai số

Mục tiêu của huấn luyện là tìm bộ tham số θ sao cho loss nhỏ nhất có thể:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\hat{y}, y)$$

Với toàn bộ tập dữ liệu:

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f(x_i; \theta), y_i)$$

Mục tiêu là:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

Chương 5

Lan truyền ngược sai số

5.1 Backpropagation là gì?

Lan truyền ngược sai số, tiếng Anh là *backpropagation*, là kỹ thuật dùng sai số ở output để điều chỉnh các tham số bên trong mạng.

Quá trình tổng quát:

Input $\rightarrow$ Lan truyền tiến $\rightarrow$ Output dự đoán

So sánh với label $\rightarrow$ Tính loss

Lan truyền ngược sai số $\rightarrow$ Cập nhật trọng số và bias

Ghi nhớ

Backpropagation là một trong những kỹ thuật phổ biến nhất để huấn luyện mạng nơ-ron hiện nay, nhưng không phải là kỹ thuật duy nhất.

5.2 Ý tưởng trực quan

Ban đầu, mạng giống như một đứa trẻ chưa biết làm gì. Các trọng số được khởi tạo ngẫu nhiên nên mạng trả lời sai. Mỗi khi mạng sai, ta dùng mức độ sai đó để chỉnh lại các trọng số một chút.

Sau rất nhiều lần chỉnh như vậy, mạng dần học được cách tạo output gần với label đúng.

Ví dụ minh họa

Quá trình học qua ba mẫu dữ liệu:

1. Đưa ảnh con chó vào. Mạng đoán “máy bay”. So với label “chó”, mạng sai. Ta cập nhật trọng số.
2. Đưa ảnh con mèo vào. Mạng đoán “xe tải”. So với label “mèo”, mạng sai. Ta tiếp tục cập nhật trọng số.
3. Đưa ảnh con voi vào. Mạng đoán “ngôi nhà”. So với label “voi”, mạng sai. Ta lại cập nhật trọng số.

Lặp lại quá trình này với rất nhiều mẫu dữ liệu, mạng có cơ hội học dần quan hệ giữa ảnh và nhãn.

5.3 Cập nhật từng bước

Giả sử tại bước huấn luyện thứ t , bộ tham số của mạng là:

$$\theta_t$$

Sau khi tính loss, thuật toán cập nhật tham số:

$$\theta_{t+1} = \theta_t + \Delta\theta_t$$

Trong thực tế, nếu dùng gradient descent, công thức thường có dạng:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

Trong đó:

- η là tốc độ học, tiếng Anh là *learning rate*;
- $\nabla_{\theta} J(\theta_t)$ là gradient của hàm mất mát theo tham số;
- dấu trừ thể hiện việc đi theo hướng làm giảm loss.

Lưu ý

Trong transcript, giảng viên chủ yếu trình bày ý tưởng tổng quát: dùng sai số giữa output hiện tại và output mong muốn để quay lại điều chỉnh các trọng số. Để lập trình được backpropagation, cần học thêm các chi tiết toán học như đạo hàm riêng, gradient, chain rule và thuật toán tối ưu.

5.4 Liên hệ với điều khiển tự động

Ý tưởng dùng sai lệch giữa kết quả mong muốn và kết quả hiện tại để điều chỉnh hệ thống có liên hệ với tư duy trong điều khiển tự động.

Trong điều khiển phản hồi:

$$e = r - y$$

Trong đó:

- r là giá trị mong muốn;
- y là giá trị thực tế;
- e là sai lệch.

Hệ thống dùng sai lệch e để điều chỉnh tín hiệu điều khiển.

Trong huấn luyện mạng nơ-ron:

$$\text{sai số} = \text{label đúng} - \text{output dự đoán}$$

Mạng dùng sai số này để điều chỉnh trọng số và bias.

Chương 6

Quá trình huấn luyện lặp

6.1 Huấn luyện qua từng mẫu

Với mỗi cặp dữ liệu (x_i, y_i) , ta làm các bước:

1. Đưa x_i vào mạng.
2. Lan truyền tiến để tính $\hat{y}_i$.
3. So sánh $\hat{y}_i$ với y_i .
4. Tính loss.
5. Lan truyền ngược sai số.
6. Cập nhật trọng số và bias.

Sau đó chuyển sang mẫu tiếp theo.

6.2 Epoch

Một epoch là một lượt mô hình đi qua toàn bộ tập dữ liệu huấn luyện.

Nếu tập dữ liệu có N mẫu, thì sau một epoch, mạng đã được huấn luyện trên N mẫu đó một lần.

Thông thường, ta cần nhiều epoch:

Training = nhiều epoch

6.3 Vì sao cần lặp nhiều lần?

Một lần cập nhật thường chỉ làm thay đổi tham số một chút. Vì vậy, mạng cần rất nhiều lần cập nhật để học được một quan hệ phức tạp.

Ví dụ minh họa

Nếu mạng có hàng triệu tham số và dữ liệu gồm hàng trăm nghìn ảnh, việc chỉ học một vài ảnh là không đủ. Mạng cần nhìn thấy nhiều ảnh chó, nhiều ảnh mèo, nhiều ảnh xe hơi trong nhiều điều kiện khác nhau để học được đặc trưng tổng quát.

6.4 Thời gian huấn luyện

Thời gian huấn luyện phụ thuộc vào:

- kích thước tập dữ liệu;
- độ phức tạp của mạng;
- số lượng tham số;
- số epoch;
- cấu hình phần cứng;
- loại GPU hoặc card màn hình;
- cách cài đặt thuật toán.

Lưu ý

Huấn luyện mạng nơ-ron có thể mất vài giờ, vài ngày hoặc thậm chí vài tuần. Với các mô hình lớn, phần cứng mạnh, đặc biệt là GPU, đóng vai trò rất quan trọng.

Chương 7

Bài toán phân loại nhiều lớp

7.1 Khái niệm class

Trong bài toán phân loại, *class* là lớp dữ liệu hoặc nhóm nhãn mà mô hình cần dự đoán. Ví dụ bài toán có 4 class:

{chó, mèo, điều, xe hơi}

Mỗi ảnh đầu vào được gán thuộc đúng một class.

Ghi nhớ

Nếu bài toán yêu cầu mỗi mẫu chỉ thuộc một lớp duy nhất, ta gọi đó là bài toán phân loại một nhãn, tiếng Anh là *single-label classification*.

7.2 Thiết kế output cho bài toán nhiều lớp

Với 4 class, có nhiều cách thiết kế output.

7.2.1 Cách 1: Một output nhận giá trị 1, 2, 3, 4

Ta có thể quy ước:

1 = chó, 2 = mèo, 3 = điều, 4 = xe hơi

Khi đó mạng chỉ có một output.

Tuy nhiên, cách này có vấn đề: các con số 1, 2, 3, 4 có thứ tự số học, nhưng các class không nhất thiết có quan hệ thứ tự. Ví dụ, không có nghĩa là “xe hơi” lớn hơn “chó” theo nghĩa toán học.

7.2.2 Cách 2: Hai output theo mã nhị phân

Với 4 class, ta có thể dùng 2 bit:

00, 01, 10, 11

Ví dụ:

00 = chó, 01 = mèo, 10 = điều, 11 = xe hơi

Cách này tiết kiệm số output hơn nhưng không phải cách phổ biến nhất trong bài toán phân loại cơ bản.

7.2.3 Cách 3: Bốn output theo one-hot vector

Cách phổ biến là dùng số output bằng số class. Với 4 class, ta dùng 4 output.

Mỗi label được biểu diễn bằng một vector có đúng một phần tử bằng 1, các phần tử còn lại bằng 0.

$$\text{chó} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \text{mèo} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \text{điều} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \text{xe hơi} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Ghi nhớ

One-hot vector là cách biểu diễn nhãn rất phổ biến trong bài toán phân loại nhiều lớp.

Chương 8

One-hot encoding

8.1 One-hot vector là gì?

One-hot vector là vector trong đó chỉ có một phần tử mang giá trị 1, các phần tử còn lại mang giá trị 0.

Nếu có K class, one-hot vector sẽ có K phần tử.

Ví dụ với $K = 4$:

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

Vector này biểu diễn class thứ 2.

8.2 Bảng mã hóa one-hot

Class	Vector one-hot	Ý nghĩa
Chó	$[1, 0, 0, 0]^T$	Phần tử thứ nhất bằng 1
Mèo	$[0, 1, 0, 0]^T$	Phần tử thứ hai bằng 1
Điêu	$[0, 0, 1, 0]^T$	Phần tử thứ ba bằng 1
Xe hơi	$[0, 0, 0, 1]^T$	Phần tử thứ tư bằng 1

8.3 Output dự đoán của mạng

Trong thực tế, output của mạng thường không ra đúng tuyệt đối các giá trị 0 và 1. Nó có thể ra một vector điểm số hoặc xác suất.

Ví dụ, với ảnh con chó:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Nhưng mạng dự đoán:

$$\hat{y} = \begin{bmatrix} 0.70 \\ 0.20 \\ 0.05 \\ 0.05 \end{bmatrix}$$

Ta có thể hiểu mạng đang cho rằng:

- xác suất là chó: 70%;
- xác suất là mèo: 20%;
- xác suất là diều: 5%;
- xác suất là xe hơi: 5%.

Dự đoán cuối cùng là class có giá trị lớn nhất:

$$\arg \max(\hat{y}) = \text{chó}$$

Ví dụ minh họa

Nếu mạng cho output:

$$\hat{y} = \begin{bmatrix} 0.1 \\ 0.6 \\ 0.2 \\ 0.1 \end{bmatrix}$$

thì phần tử lớn nhất là 0.6, nằm ở vị trí thứ 2. Mạng dự đoán ảnh thuộc class thứ 2, tức là “mèo”.

Chương 9

Đo sai số giữa hai vector

9.1 Vì sao cần đo sai số giữa hai vector?

Trong bài toán phân loại nhiều lớp, label đúng thường là một vector one-hot, còn output dự đoán cũng là một vector.

Ví dụ:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Ta cần một cách đo sự khác biệt giữa hai vector này.

Ghi nhớ

Khi output và label đều là vector, sai số không còn đơn giản là phép trừ giữa hai số. Ta cần dùng các đại lượng như chuẩn vector, góc giữa hai vector hoặc cross entropy.

Chương 10

Hàm mất mát dựa trên chuẩn L2 - NORM 2

10.1 Chuẩn L2 của vector

Với vector:

$$v = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

Chuẩn L_2 của v là:

$$\|v\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

Đây chính là độ dài Euclid của vector.

10.2 Sai số L2 giữa hai vector

Với label đúng y và output dự đoán $\hat{y}$, vector sai số là:

$$e = y - \hat{y}$$

Sai số theo chuẩn L_2 là:

$$\|y - \hat{y}\|_2$$

Hoặc thường dùng bình phương chuẩn L_2 :

$$\|y - \hat{y}\|_2^2$$

10.3 Ví dụ tính sai số L2

Giả sử:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Khi đó:

$$e = y - \hat{y} = \begin{bmatrix} 1 - 0.7 \\ 0 - 0.1 \\ 0 - 0.1 \\ 0 - 0.1 \end{bmatrix} = \begin{bmatrix} 0.3 \\ -0.1 \\ -0.1 \\ -0.1 \end{bmatrix}$$

Chuẩn L_2 :

$$\|e\|_2 = \sqrt{0.3^2 + (-0.1)^2 + (-0.1)^2 + (-0.1)^2}$$

$$\|e\|_2 = \sqrt{0.09 + 0.01 + 0.01 + 0.01} = \sqrt{0.12} \approx 0.346$$

Bình phương chuẩn L_2 :

$$\|e\|_2^2 = 0.12$$

Ghi nhớ

Sai số L_2 càng nhỏ thì hai vector càng gần nhau.

Chương 11

Đo khác biệt bằng góc giữa hai vector

11.1 Tích vô hướng

Với hai vector:

$$p = \begin{bmatrix} p_1 \\ p_2 \\ \vdots \\ p_n \end{bmatrix}, \quad q = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{bmatrix}$$

Tích vô hướng là:

$$p \cdot q = p_1q_1 + p_2q_2 + \cdots + p_nq_n$$

Tích vô hướng cho ra một số vô hướng, không phải một vector.

Lưu ý

Không nhầm tích vô hướng với tích có hướng. Tích vô hướng giữa hai vector cho ra một số thực. Tích có hướng mới cho ra một vector trong không gian ba chiều.

11.2 Góc giữa hai vector

Góc θ giữa hai vector p và q được tính bởi:

$$\cos \theta = \frac{p \cdot q}{\|p\|_2 \|q\|_2}$$

Suy ra:

$$\theta = \arccos \left(\frac{p \cdot q}{\|p\|_2 \|q\|_2} \right)$$

11.3 Ý nghĩa

- Nếu θ gần 0° , hai vector gần cùng hướng, tức là tương đồng cao.
- Nếu θ gần 90° , hai vector gần vuông góc, tức là ít liên quan theo hướng.
- Nếu θ gần 180° , hai vector gần ngược hướng.

Ví dụ minh họa

Cho:

$$p = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad q = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Ta có:

$$p \cdot q = 1 \cdot 0 + 0 \cdot 1 = 0$$

$$\|p\|_2 = 1, \quad \|q\|_2 = 1$$

Do đó:

$$\cos \theta = \frac{0}{1 \cdot 1} = 0$$

$$\theta = 90^\circ$$

Hai vector vuông góc với nhau.

11.4 Cosine similarity

Trong nhiều ứng dụng, thay vì dùng trực tiếp góc, ta dùng cosine similarity:

$$\text{cosine similarity}(p, q) = \frac{p \cdot q}{\|p\|_2 \|q\|_2}$$

Giá trị này càng gần 1 thì hai vector càng giống nhau về hướng.
Một dạng loss có thể xây dựng từ cosine similarity là:

$$\mathcal{L}_{\cos} = 1 - \frac{p \cdot q}{\|p\|_2 \|q\|_2}$$

Chương 12

Cross entropy

12.1 Ý tưởng của cross entropy

Cross entropy là một hàm mất mát rất phổ biến trong bài toán phân loại.

Giả sử:

p = phân phối đúng

q = phân phối dự đoán

Cross entropy giữa p và q được định nghĩa:

$$H(p, q) = - \sum_{i=1}^K p_i \log(q_i)$$

Trong đó:

- K là số class;
- p_i là xác suất đúng của class i ;
- q_i là xác suất mô hình dự đoán cho class i .

12.2 Cross entropy với one-hot label

Nếu label đúng là one-hot vector, chỉ có một phần tử $p_c = 1$, các phần tử còn lại bằng 0.

Khi đó:

$$H(p, q) = -\log(q_c)$$

Trong đó q_c là xác suất mô hình gán cho class đúng.

Ghi nhớ

Với one-hot label, cross entropy chỉ quan tâm trực tiếp đến xác suất mà mô hình gán cho class đúng. Nếu mô hình gán xác suất cao cho class đúng, loss nhỏ. Nếu mô hình gán xác suất thấp cho class đúng, loss lớn.

12.3 Ví dụ tính cross entropy

Giả sử có 4 class:

{chó, mèo, điều, xe hơi}

Ảnh thật là chó, nên:

$$p = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

Mạng dự đoán:

$$q = \begin{bmatrix} 0.7 \\ 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Cross entropy:

$$H(p, q) = -[1 \log(0.7) + 0 \log(0.1) + 0 \log(0.1) + 0 \log(0.1)]$$

$$H(p, q) = -\log(0.7) \approx 0.357$$

Nếu mạng dự đoán tệ hơn:

$$q = \begin{bmatrix} 0.1 \\ 0.6 \\ 0.2 \\ 0.1 \end{bmatrix}$$

thì:

$$H(p, q) = -\log(0.1) \approx 2.303$$

Loss lớn hơn nhiều vì mô hình chỉ gán xác suất 10% cho class đúng.

12.4 So sánh các cách đo sai số

Cách đo	Ý tưởng	Thường dùng khi
Chuẩn L_2	Đo độ dài vector sai số $y - \hat{y}$	Bài toán hồi quy hoặc so sánh vector đơn giản
Góc giữa hai vector	Đo sự giống nhau về hướng	Bài toán so sánh độ tương đồng vector
Cross entropy	Đo sai khác giữa phân phối đúng và phân phối dự đoán	Bài toán phân loại nhiều lớp

Chương 13

Mô hình học được những gì?

13.1 Mô hình chỉ học trong phạm vi dữ liệu được dạy

Một điểm quan trọng trong transcript là: mô hình học những gì nó được dạy.

Nếu tập dữ liệu chỉ gồm 10 loại con vật, mô hình chỉ được học để phân biệt 10 loại đó. Nếu đưa vào một con vật thứ 11 chưa từng xuất hiện trong dữ liệu huấn luyện, mô hình có thể trả lời sai hoặc trả lời một cách không đáng tin cậy.

Ví dụ minh họa

Giả sử mô hình được huấn luyện với 4 class:

{chó, mèo, điều, xe hơi}

Nếu đưa ảnh một con voi vào, nhưng mô hình không có class “voi”, nó vẫn có thể buộc phải chọn một trong 4 class đã biết. Ví dụ, nó có thể dự đoán nhầm là “chó” hoặc “xe hơi”.

13.2 Số lượng class trong các bộ dữ liệu lớn

Trong thực tế, nhiều mô hình phân loại ảnh được huấn luyện trên các tập dữ liệu có rất nhiều class. Ví dụ, một bộ dữ liệu có thể có 1000 class, bao gồm nhiều loại động vật, phương tiện, đồ vật và cảnh vật khác nhau.

Khi số class tăng, output layer cũng thường tăng số nơ-ron tương ứng nếu dùng one-hot encoding.

$$K \text{ class} \Rightarrow K \text{ output}$$

13.3 Không phải cứ huấn luyện là thành công

Huấn luyện mạng nơ-ron không đảm bảo chắc chắn mô hình sẽ tốt. Kết quả phụ thuộc vào nhiều yếu tố:

- dữ liệu có đủ lớn không;
- dữ liệu có đúng và sạch không;

- nhãn có chính xác không;
- kiến trúc mạng có phù hợp không;
- hàm mất mát có phù hợp không;
- tốc độ học có phù hợp không;
- số epoch có đủ không;
- có bị overfitting hay underfitting không.

Cảnh báo

Nếu chỉ hiểu ý tưởng tổng quát của backpropagation mà không hiểu các chi tiết toán học và kỹ thuật, việc tự lập trình hoặc huấn luyện mô hình phức tạp rất dễ thất bại.

Chương 14

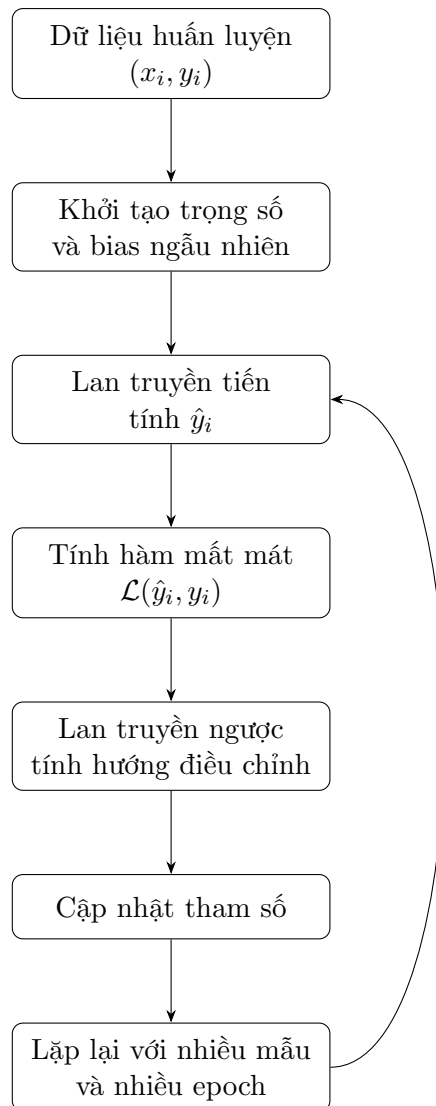
Tổng hợp quy trình huấn luyện ANN

14.1 Quy trình tổng quát

Một quy trình huấn luyện ANN có thể được mô tả như sau:

1. Chuẩn bị tập dữ liệu gồm các cặp (x_i, y_i) .
2. Thiết kế kiến trúc mạng.
3. Chọn hàm kích hoạt.
4. Chọn cách biểu diễn output, ví dụ one-hot vector.
5. Chọn hàm mất mát.
6. Khởi tạo ngẫu nhiên trọng số và bias.
7. Thực hiện lan truyền tiến để tính output dự đoán.
8. Tính loss giữa output dự đoán và label đúng.
9. Dùng backpropagation để tính hướng điều chỉnh tham số.
10. Cập nhật tham số.
11. Lặp lại quá trình trên nhiều lần cho đến khi mô hình đạt chất lượng mong muốn.

14.2 Sơ đồ tổng quát



Chương 15

Bảng thuật ngữ chuyên ngành

Thuật ngữ	Giải thích	Ví dụ
ANN	Artificial Neural Network, mạng nơ-ron nhân tạo.	Mạng phân loại ảnh.
Training	Quá trình huấn luyện mô hình bằng dữ liệu.	Cho mạng học từ ảnh và nhãn.
Supervised learning	Học có giám sát, mô hình học từ các cặp input-label.	Ảnh chó đi kèm nhãn “chó”.
Input	Dữ liệu đầu vào của mạng.	Một ảnh, một vector giá cổ phiếu.
Label	Nhãn đúng của dữ liệu.	Chó, mèo, xe hơi.
Target	Đầu ra mong muốn, thường tương đương label.	Vector $[1, 0, 0, 0]^T$.
Output	Kết quả mà mạng tạo ra.	$\hat{y} = [0.7, 0.1, 0.1, 0.1]^T$.
Parameter	Tham số được học từ dữ liệu.	Trọng số w và bias b .
Weight	Trọng số, biểu diễn mức ảnh hưởng của một kết nối.	$w_1 = 0.5$.
Bias	Hằng số cộng thêm vào tổng có trọng số.	$z = wx + b$.
Random initialization	Khởi tạo tham số ban đầu bằng giá trị ngẫu nhiên.	Trọng số ban đầu lấy ngẫu nhiên.
Forward propagation	Lan truyền tiến từ input đến output.	Tính lần lượt qua các lớp.
Backpropagation	Lan truyền ngược sai số để cập nhật tham số.	Dùng loss để chỉnh trọng số.
Loss function	Hàm mất mát, đo sai khác giữa dự đoán và label.	Cross entropy.
Error	Sai số giữa output dự đoán và output mong muốn.	$e = y - \hat{y}$.
Gradient	Vector đạo hàm cho biết hướng tăng nhanh nhất của loss.	$\nabla_{\theta} J(\theta)$.
Learning rate	Tốc độ học, quyết định bước cập nhật tham số lớn hay nhỏ.	$\eta = 0.001$.
Epoch	Một lượt đi qua toàn bộ tập dữ liệu huấn luyện.	Huấn luyện 50 epoch.
Class	Lớp dữ liệu trong bài toán phân loại.	Chó, mèo, xe hơi.

Thuật ngữ	Giải thích	Ví dụ
Classification	Bài toán phân loại dữ liệu vào các class.	Phân loại ảnh động vật.
One-hot vector	Vector có đúng một phần tử bằng 1, còn lại bằng 0.	$[0, 1, 0, 0]^T$.
One-hot encoding	Cách mã hóa nhãn bằng one-hot vector.	Mèo $\rightarrow [0, 1, 0, 0]^T$.
L_2 norm	Chuẩn Euclid của vector.	$\ v\ _2 = \sqrt{\sum v_i^2}$.
Dot product	Tích vô hướng giữa hai vector, cho ra một số.	$p \cdot q = \sum p_i q_i$.
Cosine similarity	Độ tương đồng theo góc giữa hai vector.	$\frac{p \cdot q}{\ p\ \ q\ }$.
Cross entropy	Hàm mất mát phổ biến cho phân loại.	$-\sum p_i \log q_i$.
GPU	Bộ xử lý đồ họa, thường dùng để tăng tốc huấn luyện mạng lớn.	Card màn hình NVIDIA.

Chương 16

Tóm tắt kiến thức trọng tâm

16.1 Các ý chính cần nhớ

1. Ban đầu, mạng nơ-ron chưa biết suy luận vì các tham số được khởi tạo ngẫu nhiên.
2. Khi đưa input vào mạng, mạng thực hiện lan truyền tiến để tạo output dự đoán.
3. Output dự đoán được so sánh với label đúng để tính sai số.
4. Hàm mất mát biến sai số thành một số vô hướng.
5. Backpropagation dùng sai số này để quay ngược lại điều chỉnh các trọng số và bias.
6. Quá trình cập nhật tham số được lặp lại rất nhiều lần.
7. Trong phân loại nhiều lớp, label thường được biểu diễn bằng one-hot vector.
8. Các cách đo sai số giữa vector gồm chuẩn L_2 , góc giữa hai vector và cross entropy.
9. Cross entropy đặc biệt phổ biến trong bài toán phân loại.
10. Mô hình chỉ học tốt những gì được thể hiện đủ rõ và đủ đúng trong dữ liệu huấn luyện.

16.2 Công thức quan trọng

Lan truyền tiến qua một lớp

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = \varphi(z^{(l)})$$

Sai số vector

$$e = y - \hat{y}$$

Chuẩn L2

$$\|e\|_2 = \sqrt{e_1^2 + e_2^2 + \dots + e_n^2}$$

Góc giữa hai vector

$$\cos \theta = \frac{p \cdot q}{\|p\|_2 \|q\|_2}$$

Cross entropy

$$H(p, q) = - \sum_{i=1}^K p_i \log(q_i)$$

Cập nhật tham số theo gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

Chương 17

Câu hỏi ôn tập

1. Vì sao mạng nơ-ron ban đầu thường cho output sai?
2. Khởi tạo ngẫu nhiên trọng số có ý nghĩa gì?
3. Lan truyền tiến là gì?
4. Backpropagation là gì?
5. Vì sao cần hàm mất mát?
6. Hàm mất mát phải trả về vector hay số vô hướng?
7. One-hot vector là gì?
8. Với 5 class, one-hot vector có bao nhiêu phần tử?
9. Viết công thức chuẩn L_2 của một vector.
10. Tích vô hướng giữa hai vector cho ra số hay vector?
11. Góc giữa hai vector cho biết điều gì?
12. Cross entropy thường dùng cho loại bài toán nào?
13. Vì sao mô hình có thể trả lời sai khi gặp class chưa từng được học?
14. Vì sao huấn luyện mạng nơ-ron có thể mất nhiều giờ hoặc nhiều ngày?
15. Vì sao chỉ hiểu ý tưởng backpropagation là chưa đủ để lập trình hoàn chỉnh?

Chương 18

Bài tập vận dụng

Bài tập 1: One-hot encoding

Cho 5 class:

{chó, mèo, chim, xe hơi, máy bay}

Hãy viết one-hot vector cho từng class theo đúng thứ tự trên.

Lời giải gợi ý

$$\begin{aligned} \text{chó} &= \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, & \text{mèo} &= \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \\ \text{chim} &= \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, & \text{xe hơi} &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, & \text{máy bay} &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \end{aligned}$$

Bài tập 2: Tính chuẩn L2

Cho:

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \hat{y} = \begin{bmatrix} 0.8 \\ 0.1 \\ 0.1 \end{bmatrix}$$

Tính:

$$\|y - \hat{y}\|_2$$

Lời giải gợi ý

$$e = y - \hat{y} = \begin{bmatrix} 0.2 \\ -0.1 \\ -0.1 \end{bmatrix}$$

$$\|e\|_2 = \sqrt{0.2^2 + (-0.1)^2 + (-0.1)^2}$$

$$\|e\|_2 = \sqrt{0.04 + 0.01 + 0.01} = \sqrt{0.06} \approx 0.245$$

Bài tập 3: Tính cross entropy

Cho label đúng:

$$p = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Mạng dự đoán:

$$q = \begin{bmatrix} 0.2 \\ 0.7 \\ 0.1 \end{bmatrix}$$

Tính cross entropy:

$$H(p, q) = - \sum_{i=1}^3 p_i \log(q_i)$$

Lời giải gợi ý

Vì class đúng là class thứ 2:

$$H(p, q) = -\log(0.7) \approx 0.357$$

Bài tập 4: Xác định output layer

Một bài toán phân loại ảnh có 12 class. Nếu dùng one-hot encoding, lớp output nên có bao nhiêu nơ-ron?

Lời giải gợi ý

Vì có 12 class, one-hot vector có 12 phần tử. Do đó, lớp output nên có 12 nơ-ron.

$$12 \text{ class} \Rightarrow 12 \text{ output neurons}$$

Kết luận

ANN3 giải thích phần cốt lõi của quá trình huấn luyện mạng nơ-ron. Ban đầu, mạng có các trọng số và bias ngẫu nhiên nên output thường sai. Thông qua lan truyền tiến, mạng tạo dự đoán. Dự đoán này được so sánh với label đúng bằng một hàm mất mát. Sau đó, backpropagation dùng sai số để điều chỉnh các tham số của mạng.

Trong bài toán phân loại nhiều lớp, nhãn thường được mã hóa bằng one-hot vector. Khi đó, output và label đều là vector, nên cần các cách đo sai số giữa vector như chuẩn L_2 , góc giữa hai vector hoặc cross entropy. Trong thực tế, cross entropy là lựa chọn rất phổ biến cho bài toán phân loại.

Điều quan trọng cần nhớ là huấn luyện mạng nơ-ron không chỉ là chạy một thuật toán có sẵn. Muốn mô hình hoạt động tốt, người học cần hiểu dữ liệu, cách mã hóa nhãn, hàm mất mát, thuật toán tối ưu, kiến trúc mạng và nhiều chi tiết kỹ thuật khác.